

1. Comparison Internal sorting vs External sorting.

* Internal sorting:-

- Entire dataset is loaded into main memory (RAM)
- High memory requirement
- It is suitable for small to medium datasets that can fit in memory
- Typically handles up to RAM capacity (GBs)
- faster due to an memory operations
- lower I/O operations
- performance depends on CPU speed and memory bandwidth
- limited by the size of available RAM

* External sorting:-

- only a portion of data is loaded into memory at a time
- Low memory requirement
- uses disk storage for the remaining data
- suitable for very large datasets that do not fit in memory
- it can handle data of terabytes (TBs) or even petabytes (PBs)
- slower due to high I/O operations (disk read/write)
- performance depends on disk speed, I/O bandwidth and algorithm efficiency

3 Comparison: parallel merge sort vs sequential merge:-

* parallel mergesort:-

- it divides the sorting and merging tasks among multiple processors / cores
- significantly reduces execution time.
- Time Complexity $O(n \log n / p)$
(p = no. of processors)
- Has Overhead due to;
 - * thread/process creation
 - * communication b/w processors
 - * data synchronization
- Overhead increases with more processors
- is highly suitable for large datasets
- scales well with increasing data size

* Sequential merge sort:-

- uses a single processor to perform sorting & merging
- Takes more time as data size increases
- Time Complexity: $O(n \log n)$
- No Overhead due to parallelism
- simple implementation with no communication or synchronization cost
- suitable for small to medium datasets
- becomes slow & inefficient for very large datasets
- limited by single CPU & memory

Shif
14/08/2026